

Optimization for Data Science

Lecture 4: Unconstrained Methods, Line Search, and Quasi-Newton

Prof. Udit Bhatia

Indian Institute of Technology Gandhinagar

bhatia.u@iitgn.ac.in

January 29, 2025

Outline

- 1 Recap: Unconstrained Optimization
- 2 Motivation & Intuition (High-School Level)
- 3 Armijo Backtracking (Theory & Examples)
- 4 Armijo: 1D Example (Fully Worked)
- 5 Armijo: 2D Example
- 6 Wolfe Conditions
- 7 Wolfe: 1D Example
- 8 Quasi-Newton: Detailed Explanation
- 9 Conclusions

(1) The Setting: Unconstrained Optimization

General Problem:

$$\min_{\mathbf{w} \in \mathbb{R}^d} f(\mathbf{w}),$$

where $\mathbf{w}$ is unrestricted in $\mathbb{R}^d$ (no explicit constraints).

Data Science Examples:

- *Least Squares* regression: $\mathbf{w}$ are coefficients.
- *Logistic Regression*: $\mathbf{w}$ are classifier parameters.
- *Neural Networks*: $\mathbf{w}$ represents all weights unless we impose constraints.

Optimality (smooth f):

$$\nabla f(\mathbf{w}^*) = \mathbf{0}, \quad \nabla^2 f(\mathbf{w}^*) \succeq 0 \quad (\text{if local min}).$$

If f is convex, this is a global minimum.

(2) Gradient Descent Recap

Gradient Descent Update:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \alpha \nabla f(\mathbf{w}^{(k)}).$$

Interpretation:

- We move in the direction $-\nabla f(\mathbf{w}^{(k)})$, the steepest descent direction.
- $\alpha > 0$ is the step size (learning rate).

Key Issue:

- If α is too large, we might overshoot.
- If α is too small, convergence is very slow.

Hence the motivation for systematically choosing α via *line search*.

$\alpha = ??$

→ 0.01

→ 0.005

→ shouldn't be
cons ~ see
varying?

(3) Newton's Method Recap

Newton's Idea: Use second-order information (the Hessian) for a local quadratic approximation.

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - (\nabla^2 f(\mathbf{w}^{(k)}))^{-1} \nabla f(\mathbf{w}^{(k)}).$$

Advantages:

- Quadratic convergence near the optimum if Hessian is positive definite.
- Often fewer iterations than gradient descent.

Drawback:

- Forming/inverting the Hessian is expensive for large d .
- A "full" Newton step might be too large if far from optimum, so we often do a line search or trust region approach.

(4) Example: Quadratic $f(\mathbf{w})$

For a **perfect quadratic**, Newton's method can find the solution in one step (if Hessian is invertible).

$$f(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top A \mathbf{w} - \mathbf{b}^\top \mathbf{w}, \quad \nabla f = A \mathbf{w} - \mathbf{b}, \quad \nabla^2 f = A$$

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - A^{-1}(A \mathbf{w}^{(k)} - \mathbf{b}) = A^{-1} \mathbf{b}.$$

But **real problems** are not always purely quadratic. Also A might be large.

(5) Comparing Gradient Descent and Newton



- **Gradient Descent**: simpler per iteration, but might need many iterations or careful α .
- **Newton's Method**: fewer iterations, but the Hessian cost is high for large dimension.

Hence:

- *Line Search* strategies for stable step sizes.
- *Quasi-Newton* methods to approximate Hessian (to be discussed).

We now transition to **line search methods** that systematically choose α_k each iteration.

slowing down!! ← $f''(x)$
 $f'(x)$
1/2

(6) Transition: Why Step-Size Matters

Core Problem:

- We know a “descent direction” (like $-\nabla f$ or the Newton direction), but *how far* should we go along it?

Line Search:

- Provides an *algorithmic* rule for picking α_k .
- Avoids trial-and-error with a fixed or decaying schedule.

Let's dive into the motivation with a simple analogy.

(A1) The “Hiker on a Mountain” Analogy



Imagine a hiker who wants to climb **down** a mountain to reach the valley floor:

- The *direction* of steepest descent is “downhill.”
- But how *far* should they step in that direction?

Two extremes:

- Take a *huge* step: might jump off a cliff or end up climbing the next mountain (overshoot).
- Take a *tiny* step: definitely go downhill, but might take forever to reach the bottom.

Line Search: systematically tries step sizes so the hiker finds a “just right” stride each time.



(A2) Why Not Always Take a “Full Step”?

In math terms: A *full step* might be $\alpha = 1$ if we use $-\nabla f$ as the direction.

But if the *valley is curved*, we can overshoot:

- Instead of ending up in a lower spot, we might land in a higher region on the other side of the valley.

Hence a line search checks:

- “Is the new spot actually *lower* than the old spot, by a decent amount?”
- If *no*, we shorten the step until we’re sure of making good progress downward.

(A3) Taylor Expansion in High-School Terms

We have a function $f(x)$ we want to minimize. - The *gradient* (or derivative in 1D) measures the slope.

In 1D:

$$\cancel{f(x + \alpha)} \approx \cancel{f(x) + \alpha \cdot f'(x)} \quad \frac{f(x + \alpha) - f(x)}{\alpha}$$

Here $f'(x)$ is the local slope.

But reality is curve-shaped: - The further we move from x , the more $f(x)$ can bend differently than this linear guess. - Armijo demands we get at least a certain fraction of that linear-predicted improvement to confirm the step is good.



(A4) Armijo's "Good Enough" Criterion



Armijo says: "I don't need the **full** linear decrease—just a fraction c ."

$$f(x + \alpha d) \leq f(x) + c \alpha f'(x) d, \quad (0 < c < 1).$$

In plain language:

- If the actual value at $x + \alpha d$ isn't low enough compared to that fraction of linear approximation, we try a smaller α .

Eventually, we find α giving a big enough drop.

(A5) Why Shrinking α Isn't Always Slow

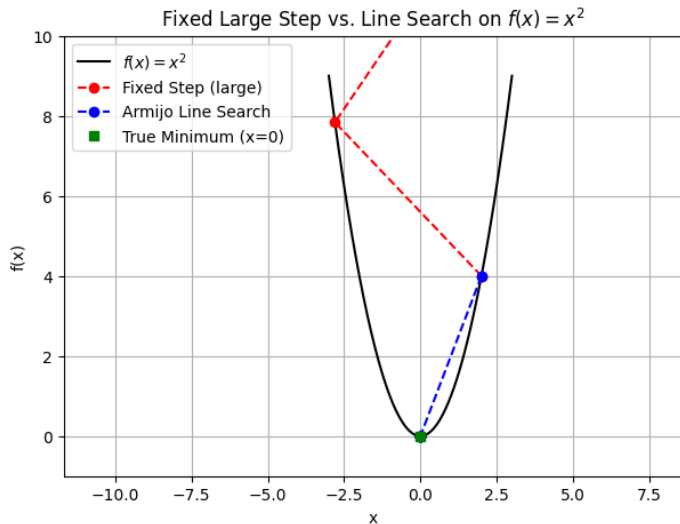
Yes, we might do multiple “tests” (halving α , for instance).

But:

- This avoids overshoot, which can be *disastrous*.
- Over many iterations, stable steps typically lead to **fewer total corrections**.

Net effect: It can be *faster overall* because we avoid big mistakes.

(A6) Illustrative Idea



(A7) Summary of Line Search Motivation (High-School Terms)

- We have a direction that's "downhill" but we're unsure how far to step.
- By systematically testing step sizes, we avoid:
 - *Overshooting* (jumping into a higher place).
 - *Undershooting* (tiny steps that barely move).
- *Armijo* or *Wolfe* line search ensures each step is in a "just-right" zone:
 - Enough decrease in f .
 - Not so small that progress is negligible.

Armijo Criterion (Theory)

Armijo (Sufficient Decrease) Condition:

* additional condⁿ x

$$f(\mathbf{x} + \alpha \mathbf{d}) \leq f(\mathbf{x}) + c \alpha \nabla f(\mathbf{x})^T \mathbf{d}, \quad 0 < c < 1.$$

Basic Idea:

$$f(\mathbf{x} + \alpha \mathbf{d}) - f(\mathbf{x}) \leq c \alpha \nabla f(\mathbf{x})^T \mathbf{d}$$

- We want the **actual decrease** in f to be at least a fraction c of the **linear** (first-order) decrease predicted by $\nabla f(\mathbf{x})^T \mathbf{d}$.
- If not, we reduce α and try again.

Armijo Backtracking Algorithm

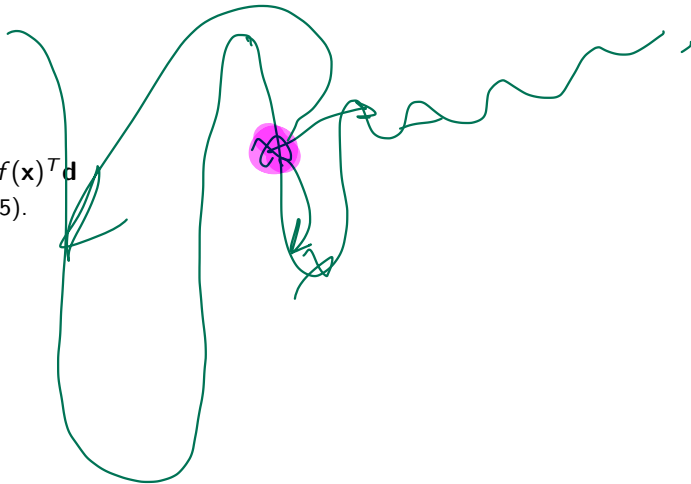
Pseudocode:

- $\alpha \leftarrow \alpha_0$ (often $\alpha_0 = 1$).
- **while** $f(\mathbf{x} + \alpha \mathbf{d}) > f(\mathbf{x}) + c \alpha \nabla f(\mathbf{x})^T \mathbf{d}$
 - $\alpha \leftarrow \beta \alpha$ ($\beta \in (0, 1)$, e.g. 0.5).

Parameters:

- $c \approx 10^{-4}$ or 10^{-3} (small).
- $\beta \approx 0.5$ or 0.8 .

Output: final α after the loop ends.



(1D Example) Function Setup

Function:

$$f(x) = (x - 3)^2 + 4.$$

- Minimum is at $x^* = 3$, with $f(3) = 4$.

Gradient:

$$f'(x) = 2(x - 3).$$

Descent Direction: (gradient descent)

$$d^{(k)} = -f'(x^{(k)}) = \underbrace{-2(x^{(k)} - 3)}_{\text{green arrow}}$$

Start from $x^{(0)} = 8$. We'll see how Armijo picks α .



Iteration 1: Gradient and Direction

$$x^{(0)} = 8, \quad f'(8) = 2(8 - 3) = 10, \quad d^{(0)} = -10.$$

Proposed update: $x^{(1)} = 8 + \alpha \times (-10)$.
Armijo parameters: $\alpha_0 = 1$, $c = 0.1$, $\beta = 0.5$.

$\underline{x_{k+1} = x_k - \nabla f(x_k) \alpha}$

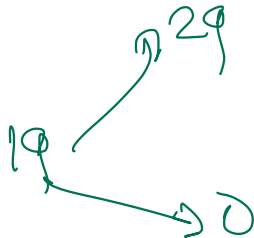
Check $\alpha = 1$



$$x_{\text{trial}} = 8 - 10 = -2, \quad f(-2) = (-5)^2 + 4 = 29,$$

$$f(8) = (5)^2 + 4 = 29, \quad 29 + 0.1 \times 1 \times 10 \times (-10) = 19.$$

Check if $29 \leq 19$? No. Condition fails. So $\alpha = 1$ is too big.



Backtrack to $\alpha = 0.5$

$$x_{\text{trial}} = 8 + 0.5 \times (-10) = 3, \quad f(3) = 4,$$

$$\text{Armijo RHS} = f(8) + 0.1 \times 0.5 \times 10 \times (-10) = 29 - 5 = 24.$$

Check if $4 \leq 24$? Yes, condition is satisfied.

Hence $\alpha = 0.5$ is accepted, $x^{(1)} = 3$.

Result

We reached $x^{(1)} = 3$, which is the global minimizer.

$$f'(3) = 0, \quad \text{so no further update.}$$

Observation:

- $\alpha = 1$ was too large.
- $\alpha = 0.5$ worked fine.
- One iteration (with a single backtrack) found the optimum.

1D Summary

- Armijo systematically tries $\alpha_0 = 1$, then shrinks α by factor β if the function decrease is insufficient.
- Freed us from guessing a single step size manually.
- In many cases, ensures stable progress each iteration.

(2D Example) Function Setup

Function:

$$f(x, y) = (x - 1)^2 + 3(y + 1)^2 + 2.$$

$$\nabla f(x, y) = \begin{pmatrix} 2(x - 1) \\ 6(y + 1) \end{pmatrix}, \quad \text{Minimum at } (1, -1).$$

Starting from $(x^{(0)}, y^{(0)}) = (2, 1)$. We do gradient descent with an Armijo line search.

Iteration 1: Calculate Gradient, Direction

$$\nabla f(2, 1) = \begin{pmatrix} 2(2 - 1) \\ 6(1 + 1) \end{pmatrix} = \begin{pmatrix} 2 \\ 12 \end{pmatrix}, \quad \mathbf{d}^{(0)} = \begin{pmatrix} -2 \\ -12 \end{pmatrix}.$$

Armijo: $\alpha_0 = 1$, $c = 0.1$, $\beta = 0.5$.

$$\mathbf{x}_{\text{trial}} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \alpha \begin{pmatrix} -2 \\ -12 \end{pmatrix}.$$

We test $\alpha = 1, 0.5, 0.25, \dots$ until the condition is satisfied.

Iteration 1: $\alpha = 1$ Check (Likely Fails)

$$\mathbf{x}_{\text{trial}} = \begin{pmatrix} 0 \\ -11 \end{pmatrix}, \quad f(0, -11) = ?$$

$$f(2, 1) = (1)^2 + 3(2)^2 + 2 = 1 + 12 + 2 = 15,$$

$$\nabla f(2, 1)^T \mathbf{d}^{(0)} = (2, 12) \cdot (-2, -12) = -4 - 144 = -148.$$

$$\text{Armijo RHS} = 15 + 0.1 \times 1 \times (-148) = 15 - 14.8 = 0.2.$$

If $f(0, -11) \gg 0.2$, the condition fails. We must reduce α .

Iteration 1: $\alpha = 0.5, 0.25, \dots$

We systematically check smaller α (like 0.5, 0.25, ...) until

$$f(\mathbf{x}^{(0)} + \alpha \mathbf{d}^{(0)}) \leq f(\mathbf{x}^{(0)}) + c \alpha \nabla f(\mathbf{x}^{(0)})^T \mathbf{d}^{(0)}.$$

Eventually, we might find $\alpha = 0.25$ (example) satisfies it. Then

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + 0.25 \mathbf{d}^{(0)}.$$

The function decreases to a moderate level (safe from overshoot).

Iteration 2

$\nabla f(\mathbf{x}^{(1)}) \rightarrow \mathbf{d}^{(1)} = -\nabla f(\mathbf{x}^{(1)})$, Again do Armijo with $\alpha_0 = 1$.

We might keep shrinking α if it overshoots, or accept it if it works.

Eventually: we converge near $(1, -1)$, the global minimizer.

Observations in 2D

- We might do multiple backtracks each iteration if the function is steep.
- Each iteration *guarantees* a “sufficient” decrease.
- Over many iterations, total performance can beat a fixed (poorly chosen) step.

Armijo in Higher Dimensions: Summary

- The logic is the same: if the function at $\mathbf{x} + \alpha \mathbf{d}$ isn't at least a fraction c better than a linear model, shrink α .
- Prevents big overshoot.
- Simple, robust approach but can be "conservative" (sometimes smaller steps than necessary).

(1) Wolfe Conditions (Overview)

Two conditions:

(i) Sufficient Decrease: same as Armijo

$$f(\mathbf{x} + \alpha \mathbf{d}) \leq f(\mathbf{x}) + c_1 \alpha \nabla f(\mathbf{x})^T \mathbf{d}.$$

(ii) Curvature Condition:

$$\nabla f(\mathbf{x} + \alpha \mathbf{d})^T \mathbf{d} \geq c_2 \nabla f(\mathbf{x})^T \mathbf{d}.$$

Parameters: $0 < c_1 < c_2 < 1$.

(2) Why Curvature Condition?

- Armijo alone might pick a step that's too small.
- The *curvature condition* ensures we don't undershoot too severely—i.e., we want the slope at the new point $\nabla f(\mathbf{x} + \alpha \mathbf{d})^T \mathbf{d}$ not to be too negative.
- If it's still very negative, we suspect we could step further.

Result: Wolfe line searches can yield bigger α in practice, improving efficiency.

(1D Example) Function Setup

Function:

$$g(x) = x^2 + 2x + 1, \quad g'(x) = 2x + 2.$$

Minimum at $x = -1$. Let's do a simple gradient descent direction from $x^{(0)} = 2$:

$$d^{(0)} = -g'(2) = -(2 \times 2 + 2) = -6.$$

We check α for (i) *Sufficient Decrease* and (ii) *Curvature Condition*.

Check Condition (i): Sufficient Decrease

$$x_{\text{trial}} = 2 + \alpha \times (-6) = 2 - 6\alpha.$$

We want

$$g(2 - 6\alpha) \leq g(2) + c_1 \alpha g'(2) \times (-6).$$

We might start with $\alpha = 1$:

$$x_{\text{trial}} = -4, \quad g(-4) = 16 - 8 + 1 = 9, \quad g(2) = 9.$$

Check $9 \leq 9 + c_1 \times 1 \times 6 \times (-6)$, etc. If it fails, reduce α .

Check Condition (ii): Curvature

$$g'(2 - 6\alpha) \times (-6) \geq c_2 g'(2) \times (-6).$$

$$g'(x) = 2x + 2, \quad g'(2) = 6.$$

So we require

$$(2(2 - 6\alpha) + 2) \times (-6) \geq c_2 \times 6 \times (-6).$$

This sets a lower bound on α (so we don't pick α too small).

Possible Outcome

- If $\alpha = 1$ fails the first condition, we reduce to $\alpha = 0.5$, check again.
- If it passes condition (i) but fails condition (ii) (curvature), we might *increase* α or search differently.
- In practice, Wolfe line search uses bracketed intervals and secant/bisection steps to find an α satisfying both conditions.

Result: A balanced step ensuring *sufficient decrease* but not too small a step.

Wolfe in Practice

- More complex to implement than pure backtracking Armijo (since we also check slope at the trial point).
- Often used with **Conjugate Gradient** or **Quasi-Newton** methods to maintain nice theoretical properties (like positive definiteness in approximate Hessians).
- Typically yields fewer “backtracks” if done carefully.

Overview and Motivation for Quasi-Newton

The Puzzle:

- **Gradient Descent:** Simple, but can be slow (linear convergence).
- **Newton's Method:** Potentially very fast (quadratic convergence) but expensive (need $\nabla^2 f$ each iteration, plus inversion or factorization).

Quasi-Newton Methods:

- *Approximate* the Hessian (or its inverse) using **only** gradient information (no need to compute $\nabla^2 f$).
- Achieve **faster convergence** than gradient descent, at a fraction of the cost of full Newton.
- Typically rely on **line search** (Armijo or Wolfe) to ensure stability and good updates.

Secant Condition: The Core Idea

Secant Condition (in scalar form) means:

$$f'(x + \Delta x) - f'(x) \approx f''(x) \Delta x.$$

- In multiple dimensions, the Hessian $\nabla^2 f$ generalizes $f''(x)$.

Hence for a *vector* step $\Delta \mathbf{s}_k$:

$$\nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)}) \approx \nabla^2 f(\mathbf{x}^{(k)}) \Delta \mathbf{s}_k.$$

If we let $B_k \approx \nabla^2 f(\mathbf{x}^{(k)})$, the condition becomes:

$$\Delta \mathbf{y}_k = \nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)}) \approx B_k \Delta \mathbf{s}_k.$$

Interpretation:

- For an **exact** Hessian, that equality would hold precisely (neglecting higher-order terms).
- Quasi-Newton updates B_{k+1} so that it *better satisfies* $\Delta \mathbf{y}_k \approx B_{k+1} \Delta \mathbf{s}_k$ after each step.

BFGS: The Classic Quasi-Newton Update

Notation:

$$B_k \approx \nabla^2 f(\mathbf{x}^{(k)}), \quad \Delta \mathbf{s}_k = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}, \quad \Delta \mathbf{y}_k = \nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)}).$$

BFGS Formula (Hessian form):

$$B_{k+1} = B_k - \frac{B_k \Delta \mathbf{s}_k (\Delta \mathbf{s}_k)^T B_k}{\Delta \mathbf{s}_k^T B_k \Delta \mathbf{s}_k} + \frac{\Delta \mathbf{y}_k (\Delta \mathbf{y}_k)^T}{\Delta \mathbf{y}_k^T \Delta \mathbf{s}_k}.$$

Why This Works:

- First (negative) term *removes* old curvature info along $\Delta \mathbf{s}_k$.
- Second (positive) term *adds* new curvature info gleaned from $\Delta \mathbf{y}_k$.
- Over iterations, B_{k+1} “learns” the local Hessian in the directions we actually move.

Why We Often Start with $B_0 = I$ (the Identity)

- If we have **no prior knowledge** about the Hessian shape, a neutral default is the identity matrix I .
- This basically says “assume unit curvature in all directions” until we gather real gradient-based evidence otherwise.
- If we suspect the Hessian is scaled, we might choose $B_0 = \alpha I$ for some scalar $\alpha > 0$.

In practice, $B_0 = I$ is extremely common. The BFGS updates quickly shape B_k into a more accurate approximation as soon as we see gradient changes $\Delta \mathbf{y}_k$ over successive steps.

Quasi-Newton Algorithmic Flow

Generic Steps:

- ① **Initialize:** $\mathbf{x}^{(0)}$ (starting guess), $B_0 = I$ (or scaled I).
- ② **For each iteration** $k = 0, 1, \dots$:
 - $\mathbf{d}^{(k)} = -B_k^{-1} \nabla f(\mathbf{x}^{(k)})$ (descent direction).
 - $\alpha_k = \text{LineSearch}(\mathbf{x}^{(k)}, \mathbf{d}^{(k)})$ (Armijo or Wolfe).
 - $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)}$.
 - $\Delta \mathbf{s}_k = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$, $\Delta \mathbf{y}_k = \nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)})$.
 - $B_{k+1} \leftarrow \text{BFGS-Update}(B_k, \Delta \mathbf{s}_k, \Delta \mathbf{y}_k)$.

Result:

- BFGS typically converges *superlinearly* near the optimum (faster than gradient descent).
- Without computing or storing the full Hessian each iteration.

A Simple 2D Example to Illustrate

Consider:

$$f(x, y) = (x - 2)^2 + 5(y + 1)^2, \quad \nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 10 \end{pmatrix} \quad (\text{constant Hessian}).$$

We pretend we *don't* know the Hessian and try to learn it from gradients only.

Setup:

- Start $\mathbf{x}^{(0)} = (0, 0)$, $B_0 = I_2$.
- $\nabla f(0, 0) = (2 \times (-2), 10 \times 1) \Rightarrow (-4, 10)$.
- $\mathbf{d}^{(0)} = -B_0^{-1} \nabla f(0, 0) = (4, -10)$.

Then do a line search for α_0 , move to $\mathbf{x}^{(1)}$, compute $\Delta \mathbf{s}_0, \Delta \mathbf{y}_0$, and update B_1 via BFGS.

After First Step: BFGS Update (Conceptual)

Move:

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha_0 \mathbf{d}^{(0)}, \quad \Delta \mathbf{s}_0 = \mathbf{x}^{(1)} - \mathbf{x}^{(0)}, \\ \Delta \mathbf{y}_0 = \nabla f(\mathbf{x}^{(1)}) - \nabla f(\mathbf{x}^{(0)}).$$

Then:

$$B_1 = B_0 - \frac{B_0 \Delta \mathbf{s}_0 (\Delta \mathbf{s}_0)^T B_0}{\Delta \mathbf{s}_0^T B_0 \Delta \mathbf{s}_0} + \frac{\Delta \mathbf{y}_0 (\Delta \mathbf{y}_0)^T}{\Delta \mathbf{y}_0^T \Delta \mathbf{s}_0}.$$

The second and third terms are *low-rank corrections* that incorporate the new information about how the gradient changed. Over more steps, B_k will approach $\begin{pmatrix} 2 & 0 \\ 0 & 10 \end{pmatrix}$.

Why the Secant Condition Improves B_k

- In an **ideal** world with a truly quadratic f , the exact Hessian satisfies $\Delta \mathbf{y}_k = \nabla^2 f \Delta \mathbf{s}_k$.
- BFGS ensures

$$B_{k+1} \Delta \mathbf{s}_k = \Delta \mathbf{y}_k,$$

(the *secant condition*) *exactly* in the updated directions, while leaving B_{k+1} as close as possible to B_k in other subspaces.

- $\Rightarrow$ We “fix” the approximation in directions where we have new gradient data, gradually refining the Hessian approximation.

Summary of Quasi-Newton Methods

- **BFGS, L-BFGS:** The most common quasi-Newton algorithms.
- **Initialization:** Often $B_0 = I$ (identity). If known scaling is useful, we choose $B_0 = \gamma I$.
- **Line Search:** Helps ensure $\Delta \mathbf{s}_k^T \Delta \mathbf{y}_k > 0$, which keeps B_{k+1} positive definite.
- **Performance:** Typically *superlinear* convergence once near optimum, bridging speed between plain gradient descent and full Newton.
- **Memory Issues for Large d :** L-BFGS uses a limited memory approach so we never store a full $d \times d$ matrix.

Line Search: Key Points

- **Armijo (Backtracking):**
 - Sufficient decrease condition only.
 - Easy to code, might be somewhat conservative.
- **Wolfe:**
 - Also checks curvature condition.
 - Balanced steps, commonly used in advanced methods.
- Both systematically pick α_k to avoid overshoot or painfully small steps.

Quasi-Newton: Key Points

- Approximates the Hessian (or inverse Hessian) to get Newton-like steps without full Hessian cost.
- **BFGS, L-BFGS** are popular variants.
- **Line Search** is vital to keep updates stable and preserve positivity in B_k .
- Achieves *superlinear* convergence typically, bridging speed between gradient descent and full Newton.

When to Use or Not Use Line Search

- **Medium-scale** problems: line search is often beneficial. E.g., logistic regression with thousands or tens of thousands of parameters.
- **Very large-scale or deep neural nets**: each function/gradient evaluation can be huge. People often prefer simpler *learning rate schedules* or *adaptive methods* (Adam, RMSProp).
- **Non-smooth problems**: subgradient or proximal methods might be needed, not the standard line search.

Practical Tips

- Choose c_1, c_2 carefully (like $c_1 = 10^{-4}$, $c_2 = 0.9$) for Wolfe.
- For Armijo, pick c small (e.g., 10^{-4}) and $\beta = 0.5$ or 0.8 .
- Initialize α_0 near 1 or based on previous iteration's α .
- Quasi-Newton methods store or update B_k or B_k^{-1} ; ensure you have a robust line search to keep B_k well-conditioned.

Example: Quasi-Newton + Line Search:

- ① Initialize $\mathbf{x}^{(0)}$, $B_0 = I$ (identity).
- ② **For** $k = 0, 1, 2, \dots$ (until convergence):
 - $\mathbf{d}^{(k)} \leftarrow -B_k^{-1} \nabla f(\mathbf{x}^{(k)})$.
 - $\alpha_k \leftarrow \text{LineSearch}(\mathbf{x}^{(k)}, \mathbf{d}^{(k)})$ (Armijo or Wolfe).
 - $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)}$.
 - Update B_{k+1} using $\Delta \mathbf{s}_k, \Delta \mathbf{y}_k$.

Overall Takeaways

- Unconstrained Optimization often tackled by iterative methods (Gradient Descent, Newton, Quasi-Newton).
- Line Search is crucial in controlling step size:
 - Armijo (simple, robust).
 - Wolfe (also checks curvature).
- Quasi-Newton merges second-order speed with less overhead than full Newton.
- For large-scale or specialized problems, further modifications or entirely different approaches may be used (SGD, Adam, etc.).

Explore Code in Colab

We have prepared an interactive notebook demonstrating the concepts in this lecture. Click the link below (or copy/paste in your browser) to experiment with optimization methods.

[Colab Notebook Link](#)

No local setup required—just run the code blocks online!

Questions?

Thank you for your attention!